Chapter 6

C Programming for Embedded Systems

Integrated Embedded Systems

MEC4126F Programming Lectures

James Hepworth

Outline

1. Introduction to C for Embedded Applications
2. **Data Elements** (Types, Scope, Qualifiers)
3. **Operators & Logic** (Arithmetic, Bitwise)
4. **Grouped Statements** (Flow Control, Functions)
5. **Structural & System Elements** (Pointers, Preprocessor)

Introduction to C

- Developed in **1972** by Dennis Ritchie at Bell Labs
- **Imperative** — statements determine program behaviour
- **Procedural** — statements execute in sequential order
- Remains the **dominant language** for embedded systems

Why C for Embedded Systems?

1. **Direct hardware access** — pointers and memory-mapped I/O
2. **Minimal runtime overhead** compared to higher-level languages
3. **Predictable performance** characteristics
4. **Fine-grained control** over system resources
5. **Portability** across different microcontroller architectures

> *C allows engineers to work closely with the hardware while maintaining readability and structure.*

1. Data Elements

(The "What")

Variables and Data Types

Data Type Categories

1. Primitive (Built-in)

- Fundamental types: `char`, `int`, `float`, `double`, `void`

2. Derived

- Built from primitives: **Arrays**, **Pointers**, **Functions**

3. User-defined

- Custom types: `struct`, `union`, `enum`, `typedef`

Variables and Data Types

Integer Types

Type	Size	Description
`char`	1 byte	Smallest addressable unit
`short`	2 bytes	Short integer
`int`	typically 4 bytes	Standard integer
`long`	4 or 8 bytes	Long integer
`long long`	8 bytes	Extra-long integer

All integer types can be **signed** (default) or **unsigned**.

Variables and Data Types

Floating-Point Types

Type	Size	Description
`float`	4 bytes	Single-precision
`double`	8 bytes	Double-precision

Note: Floating-point operations may be **expensive** without hardware FPU support.

Variables and Data Types

Character Type — `char`

- The smallest integer type (**1 byte** / 8 bits)
- **Wait, an integer?** Yes, a `char` is just a small `int`!
- Stores **numerical values** (0-255) mapped to **ASCII** (e.g. `'A'` = 65)

Context vs. `int`:

- `char`: 8 bits · Best for text or very small numbers
- `int`: 32 bits · Best for standard calculations on STM32

Fixed-Width Integer Types

In embedded C, use types from `<stdint.h>` for **consistent behaviour** across platforms:

```c
#include <stdint.h>

uint8_t  one_byte;   // Unsigned 8-bit integer
int16_t  two_bytes;  // Signed 16-bit integer
uint32_t four_bytes; // Unsigned 32-bit integer
```

- The numerical part → number of bits
- The `u` prefix → unsigned (positive only)
- No prefix → signed (positive and negative)

Integer Ranges and Limits

The range of values depends on the **number of bits (n bits)**:

Bits	Unsigned (0 to $2^n - 1$)	Signed (-2^{n-1} to $2^{n-1} - 1$)
8	0 to 255	-128 to 127
16	0 to $65,535$	$-32,768$ to $32,767$
32	0 to $\approx 4.29 \times 10^9$	$\approx \pm 2.14 \times 10^9$

Embedded Tip: Always choose the smallest type that fits your data to save memory and improve performance.

Declaration, Assignment, and Initialization

Declaration — reserves memory, value is indeterminate:

```
int counter;
float temperature;
```

Assignment — sets a value to a declared variable:

```
counter = 0;
temperature = 25.5;
```

Initialization — combines both (preferred):

```
int counter = 0;
float temperature = 25.5;
```

Multiple Declarations

Variables of the same type can be declared together:

```
uint8_t hour = 0, minute = 0, second = 0;  // Multiple init
```

> ***Warning:*** *Using a variable before assigning a value is a* ***common source of bugs****.*

Arrays

Collection of elements of the **same type** stored in contiguous memory:

```
uint16_t adc_readings[8];     // Array of 8 unsigned 16-bit integers
float temperatures[4] = {23.5, 24.1, 23.9, 25.0}; // Initialized array
```

Key Characteristics:

- **Zero-indexed:** First element is `adc_readings[0]`
- **Fixed size:** Boundary must be known at compile time
- **Contiguous memory:** Ideal for DMA and hardware buffers

> ***Warning:*** *C does NOT check* ***array bounds****! Accessing an index outside the size leads to system crashes or memory corruption.*

Accessing Arrays

```
// Assigning a value
adc_readings[0] = 4095;
adc_readings[1] = 3000;
adc_readings[2] = 2000;
adc_readings[3] = 1000;

// Accessing elements
reading_0 = adc_readings[0]; // sets reading_0 to the value of the first element in the array
reading_1 = adc_readings[1]; // sets reading_1 to the value of the second element in the array
```

Structures

Group related data of **different types** into a single unit:

```
struct SensorData
{
    uint32_t id;          // 4 bytes
    float value;         // 4 bytes
    uint16_t threshold; // 2 bytes
    char status;         // 1 byte
};
```

Usage with `struct` keyword:

```
struct SensorData reading;
reading.id = 1;
reading.value = 25.5;
reading.threshold = 100;
reading.status = 'A';
```

Custom Aliases with `typedef`

The `typedef` keyword creates a **shorter or more descriptive alias** for an existing type:

- It does **not** create a new data type; it just gives an alternative name.
- It works with **any** type: primitives, arrays, structs, or pointers.

Examples with Primitives:

```
typedef unsigned char byte_t;    // Common alias for 8-bit data
byte_t mask = 0xAA;

typedef uint32_t handle_t;      // Meaningful name for a handle
```

> ***Note:*** *The `uint32_t` and `int16_t` types you've been using are actually `typedef`s of the standard C `unsigned int` and `short` types!*

Typedef with Structures

Use `typedef` to create a shorter name for a structure to avoid typing `struct` every time:

```
typedef struct
{
    uint32_t id;
    float value;
    uint16_t threshold;
    char status;
} sensor_data_t;
```

Usage (no `struct` keyword needed!):

```
sensor_data_t reading1;
reading1.id = 1;
reading1.value = 25.5;
```

Variable Scope

Global Variables

- Declared **outside** functions
- Accessible from **anywhere** in the program
- Lives for the **entire duration** of program execution

Local Variables

- Declared **inside** a function or `{}` block
- Only accessible **within that block**
- Only exists while the block is executing

Embedded Tip: Keep globals to a minimum! They consume RAM permanently and can be altered by mistake from unrelated parts of the code.

Type Qualifiers

`const`

Declares a variable whose value **cannot be changed** after initialization:

```
const uint16_t MAX_ADC_VALUE = 4095;
const float PI = 3.14159;
```

Benefits:

1. Documents that a value should not change
2. Allows compiler to place data in **read-only memory**
3. Enables compiler **optimizations**
4. Prevents accidental modifications

Type Qualifiers

`static` — Local Variables

Preserves the variable's value **between function calls**:

```
void count_events(void)
{
    static uint32_t event_counter = 0;  // Initialized only once

    event_counter++;
    printf("Event count: %lu\n", event_counter);
}
```

Each call to `count_events()` increments from the **previous value**, not from 0.

Type Qualifiers

`static` — File Scope

Limits visibility to the file where it is defined (**encapsulation**):

```
// In timer.c
static uint32_t timer_ticks;      // Only accessible within timer.c
static void update_timer(void);   // Only callable within timer.c

void timer_init(void)             // Globally accessible
{
    timer_ticks = 0;
}
```

Benefits: Prevents name conflicts · Hides implementation details · Enforces access through public API

Type Qualifiers

`volatile`

Tells the compiler a variable's value **may change unexpectedly**:

```
volatile uint8_t interrupt_flag;
```

Critical for:

1. **Memory-mapped hardware registers**
2. Variables shared between main code and **ISRs**
3. Memory modified by **DMA** operations

Type Qualifiers

`volatile`

Tells the compiler a variable's value **may change unexpectedly**:

```
// Example: Waiting for a flag set in an Interrupt (ISR)
while (data_ready == 0)
{
    // Without volatile, the compiler might "optimize" this
    // by assuming data_ready never changes!
}
```

2. Operators & Logic

(The "How")

Arithmetic Operators

Operator	Description	Example
`+`	Addition	`a + b`
`-`	Subtraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Modulo	`a % b`
`++`	Increment	`a++` or `++a`
`--`	Decrement	`a--` or `--a`

Prefix vs. Postfix

```
uint8_t a = 5;
uint8_t b = ++a;  // b = 6, a = 6  (increment first)
uint8_t c = a++;  // c = 6, a = 7  (use first, then increment)
```

> ***Warning:*** *Division and modulo may be* ***computationally expensive*** *on microcontrollers without hardware division support.*

Assignment Operators

Operator	Description	Equivalent
`=`	Assign	—
`+=`	Add and assign	`a = a + b`
`-=`	Subtract and assign	`a = a - b`
`*=`	Multiply and assign	`a = a * b`
`/=`	Divide and assign	`a = a / b`
`%=`	Modulo and assign	`a = a % b`
`<<=`	Left shift and assign	`a = a << b`
`>>=`	Right shift and assign	`a = a >> b`
`&=`	AND and assign	`a = a & b`
`|=`	OR and assign	`a = a | b`

Comparison & Logical Operators

Comparison

Operator	Meaning
`==`	Equal to
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater/equal
`<=`	Less/equal

Logical

Operator	Meaning
`&&`	AND
`||`	OR
`!`	NOT

C uses **short-circuit evaluation**: stops evaluating as soon as the result is determined.

Examples: Logic in Practice

```
// 1. Combined conditions
if (temp > 40 && alarm_enabled)
{
    activate_fan();  // Both must be true
}

// 2. Either condition
if (button_pressed || timer_finished)
{
    wake_system();   // True if at least one is true
}

// 3. Logical NOT for flags
if (!system_busy)
{
    start_processor(); // True if system_busy is 0
}
```

Bitwise Operators

Operator	Description	Example
`&`	Bitwise AND	`a & b`
`|`	Bitwise OR	`a | b`
`^`	Bitwise XOR	`a ^ b`
`~`	Bitwise NOT	`~a`
`<<`	Left shift	`a << b`
`>>`	Right shift	`a >> b`

Heavily used in embedded systems for register manipulation.

Bitwise OR (|) — Combining Bits

Used to **combine** bit patterns or **set** bits:

```
uint8_t temp1 = 0b00000010; // (2)
uint8_t temp2 = 0b01001010; // (74)

uint8_t result = temp1 | temp2;
// result = 0b01001010 (= 0x4A = 74)
```

Bitwise AND (`&`) — Masking bits

Used to **extract** bits or **clear** bits:

```
uint8_t temp1 = 0b11111101; // (253)
uint8_t temp2 = 0b01001011; // (75)

uint8_t result = temp1 & temp2;
// result = 0b01001001 (= 0x49 = 73)
```

Bitwise XOR (`^`) — Toggling bits

Used to **toggle** (flip) bits:

```
uint8_t temp1 = 0b00000010; // (2)
uint8_t temp2 = 0b01001010; // (74)

uint8_t result = temp1 ^ temp2;
// result = 0b01001000 (= 0x48 = 72)
```

Bitwise Shift (`<<` and `>>`)

Used to move bits left or right:

```
uint8_t a = 0b00000101; // (5)

uint8_t result_left = a << 2;  // Shift left 2
// result = 0b00010100 (= 20)

uint8_t result_right = a >> 1; // Shift right 1
// result = 0b00000010 (= 2)
```

Bitwise Masking Examples

Extracting bits 0-7 from a register:

```
uint8_t low_byte = status_reg & 0xFF; // AND with 0xFF (0b11111111)
```

Clearing multiple bits at once:

```
// Example: Clearing bits 0-3 at once (0xF = 0b1111)
GPIOA->ODR &= ~(0xF << 0);
```

Checking multiple flags:

```
if ((status_reg & (FLAG_A | FLAG_B)) == (FLAG_A | FLAG_B))
{
    // Executes only if BOTH flags are set
}
```

Bitwise Operations — Common Patterns

Set a bit:

```
GPIOA->ODR |= (1 << 5);   // Set bit 5
```

Clear a bit:

```
GPIOA->ODR &= ~(1 << 5);  // Clear bit 5
```

Toggle a bit:

```
GPIOA->ODR ^= (1 << 5);   // Toggle bit 5
```

Check a bit:

```
if (GPIOA->IDR & (1 << 0)) { /* Bit 0 is set */ }
```

Miscellaneous Operators

Operator	Description	Example
`sizeof`	Size of type/variable	`sizeof(int)`
`&`	Address-of	`&variable`
`*`	Dereference pointer	`*pointer`
`.`	Structure member	`struct.member`
`->`	Structure pointer member	`ptr->member`
`?:`	Ternary conditional	`cond ? a : b`

```
uint8_t duty_cycle = (temperature > THRESHOLD) ? MAX_DUTY : MIN_DUTY;
```

A Detour to Pointers to explain the -> operator

Every variable lives at a specific location in memory — its **address**.

- A **pointer** is a variable that **stores an address** instead of a value.

```
Memory:
  Address    | Value
  ----------+---------
  0x2000     |  10       ← uint32_t val = 10;
  0x2004     |  0x2000   ← uint32_t *ptr = &val;  (stores the address of val)
```

> ***Why this matters for STM32:*** *Hardware registers like GPIO ports live at* ***fixed addresses****. The ST libraries give you pointers to those addresses so you can control hardware directly.*

Pointer Syntax

Reading pointer declarations can be confusing at first. Break it into two parts:

```
uint32_t * ptr;
│          │
│          └─ Name of the pointer variable
└─ Type of data it points to (a uint32_t lives at that address)
```

Statement	Meaning
`uint32_t val = 10;`	A 32-bit variable holding `10`
`uint32_t *ptr = &val;`	A pointer to `val`'s address
`*ptr`	The value *at* that address (i.e. `10`)

> *The `*` in a **declaration** defines a pointer. The `*` in an **expression** dereferences it.*

Referencing and Dereferencing

Two operators work together to use pointers:

Operator	Name	Meaning
`&`	Reference / **address-of**	"Give me the address of this variable"
`*`	Dereference	"Give me the value at this address"

```
uint32_t val = 10;
uint32_t *ptr = &val;    // ptr holds the address of val
uint32_t res = *ptr;     // Dereference: go to address, read value → res = 10

*ptr = 99;               // Write through the pointer: val is now 99
```

> ***Note:*** *Dereferencing a pointer is like following a hyperlink — you go to the location it points to.*

Pointers to Structures

A pointer can point to an entire **structure** — a whole block of related data — from a **single address**.

```
sensor_data_t my_sensor;          // The object in memory
sensor_data_t *ptr = &my_sensor;  // Pointer to the start of that object
```

To access a member through a pointer, you must **dereference first, then access**:

```
// Long form — dereference, then use dot operator:
(*ptr).id = 1;

// These two lines do the same thing:
my_sensor.id = 1;
(*ptr).id    = 1;
```

The Arrow Operator (`->`)

The parentheses in `(*ptr).member` are easy to get wrong. C provides a cleaner shorthand:

```
ptr->member   ≡   (*ptr).member
```

```
sensor_data_t my_sensor;
sensor_data_t *ptr = &my_sensor;

my_sensor.id = 1;   // Direct access   (object)
ptr->id      = 1;   // Arrow access     (pointer to object)  ← same result
```

Syntax	When to use
`object.member`	You have the object itself
`pointer->member`	You have a *pointer* to the object

Connecting to STM32 — Hardware Registers

All of this leads directly to how STM32 peripherals are accessed in C:

1. Registers are grouped into structs (one struct per peripheral):

```c
typedef struct {
    volatile uint32_t MODER;   // Pin mode register
    volatile uint32_t ODR;     // Output data register
    // ...
} GPIO_TypeDef;
```

2. A pointer is placed at the hardware address:

```c
#define GPIOA  ((GPIO_TypeDef *) 0x40020000UL)  // From stm32f4xx.h
```

3. You access registers with `->` :

```c
GPIOA->ODR = 0xFF;    // Write 0xFF to GPIOA's Output Data Register
```

3. Grouped Statements

(The "Logic")

Conditional Statements

`if` / `else`

```
if (ADC_value > THRESHOLD)
{
    LED_ON();
}
else
{
    LED_OFF();
}
```

`switch`

```
switch (system_state)
{
    case IDLE:    enter_low_power_mode();  break;
    case ACTIVE:  process_sensor_data();   break;
    case ERROR:   trigger_alarm();         break;
```

Loops

`for` — when number of iterations is known:

```
for (int i = 0; i < 10; i++)
{
    send_data_packet(i);
}
```

`while` — continues while condition is true:

```
while (UART_is_receiving())
{
    process_byte(UART_read_byte());
}
```

`do-while` — executes at least once:

```
do {
    read_sensor_value();
} while (more_readings_available());
```

The Embedded Super Loop

In embedded systems, `main()` typically contains an **infinite loop**:

```
int main(void)
{
    system_init();

    while (1)
    {
        if (flag_set)
        {
            process_data();
            flag_set = 0;
        }
    }
}
```

Actual processing often occurs in **interrupt service routines** (ISRs).

Functions

A well-structured C function:

```
return_type function_name(parameter_type parameter_name, ...)
{
    // Function body
    return value;
}
```

Key elements:

1. **Return type** — `void` if nothing returned
2. **Function name** — verb-noun format, e.g. `initialize_hardware`
3. **Parameters** — each needs type and name; `void` = no parameters
4. **Return statement** — must match declared return type

Functions — Example

```c
uint16_t calculate_average(uint16_t *data_array, uint8_t length)
{
    uint32_t sum = 0;

    for (uint8_t i = 0; i < length; i++)
    {
        sum += data_array[i];
    }

    return (uint16_t)(sum / length);
}
```

Uses a **pointer** parameter to avoid copying the entire array.

Function Prototypes

Declare a function **before** its implementation so the compiler can perform type checking:

```c
// Prototype - BEFORE main()
void initialize_hardware(void);

int main(void)
{
    initialize_hardware();  // Compiler knows the function exists
}

// Implementation - AFTER main()
void initialize_hardware(void)
{
    // Initialization code
}
```

4. Structural & System Elements

(The "System")

Pointers

Variables that store **memory addresses**:

```
uint32_t *ptr;              // Declare a pointer
uint32_t value = *ptr;      // Dereference — read the value
*ptr = 0x1000;              // Dereference — write a value

uint32_t variable = 42;
ptr = &variable;            // Address-of — point to variable
```

Essential for:

1. Manipulating **hardware registers** directly
2. Efficient data handling — no copying large structures
3. **Callback** mechanisms and function pointers

Pointers and Hardware Registers

Direct access to **memory-mapped registers**:

```
// Pointer to the GPIO output data register
volatile uint32_t *GPIO_ODR = (uint32_t *)0x40020014;

// Set bit 5 (turn on LED on pin 5)
*GPIO_ODR |= (1 << 5);
```

The `volatile` keyword is crucial — prevents the compiler from **optimizing away** reads/writes to hardware.

Memory-Mapped I/O

STM32 peripherals appear as **memory locations**:

```
// Base addresses
#define GPIOA_BASE    0x40020000
#define GPIOB_BASE    0x40020400

// Register offsets
#define GPIO_MODER_OFFSET  0x00
#define GPIO_ODR_OFFSET    0x14

// Pointers to specific registers
volatile uint32_t *GPIOA_MODER =
    (uint32_t *)(GPIOA_BASE + GPIO_MODER_OFFSET);
volatile uint32_t *GPIOA_ODR   =
    (uint32_t *)(GPIOA_BASE + GPIO_ODR_OFFSET);
```

Preprocessor Directives

`#include`

```
#include <stdint.h>       // System header (angle brackets)
#include "uart_driver.h"  // Project header (quotes)
```

`#define` — Macros

```
#define LED_PIN     5
#define SET_BIT(REG, BIT)  ((REG) |= (1 << (BIT)))

SET_BIT(GPIOA->ODR, LED_PIN);
```

Macros improve readability but **lack type checking**.

Conditional Compilation

Include different code sections based on defined conditions — **no runtime overhead**:

```
#define DEBUG_LEVEL 2

#if DEBUG_LEVEL > 1
    #define DEBUG_PRINT(msg)  uart_send_string(msg)
#else
    #define DEBUG_PRINT(msg)  /* nothing */
#endif
```

Useful for:

- Configurable code for **different hardware platforms**
- Enabling/disabling **debug output**
- Feature flags

Header Files

Separate **interfaces** from **implementations**:

```c
// module_name.h
#ifndef MODULE_NAME_H
#define MODULE_NAME_H

#include <stdint.h>

typedef struct {
    uint8_t id;
    uint16_t value;
} sensor_data_t;

void sensor_init(void);
sensor_data_t sensor_read(void);

#endif /* MODULE_NAME_H */
```

Include guards (`#ifndef ... #endif`) prevent multiple inclusion.

Main Program Structure

```c
// Includes  ----------------------------------------
#include <stdio.h>
#include <math.h>

// Global variables  ---------------------------------
float a = 67;

// Function declarations  ----------------------------
void main();

// Main function  ------------------------------------
void main()
{
    printf("sqrt(67)=%f", sqrt(a));
    return 0;
}

// Function implementations ---------------------------

// END  ----------------------------------------------
```

Summary

Topic	Key Takeaway
Data Elements	Use `<stdint.h>`, manage Scope, and use `volatile`
Operators & Logic	Bitwise manipulation is core for embedded
Grouped Statements	Use modular Functions and robust Control Flow
System Elements	Pointers are the bridge to the Hardware

Questions?

Chapter 6: C Programming for Embedded Systems